

CS2614 Fundamentals of Blockchain

Mini Project Report

FRAUD DETECTION AND TRANSACTION MONITORING LAYER FOR ON-CHAIN FINANCE

Submitted

By

Rashmi DT (USN0362)

Bhoomika P (USN0090)

Anita (USN0055)

Under the guidance of:

Dr. J. Cynthia

School of Computer Science and Engineering

RV University, Bangalore

RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2614 Fundamentals of Blockchain Mini Project work titled ***FRAUD DETECTION AND TRANSACTION MONITORING LAYER FOR ON-CHAIN FINANCE*** is carried out by Rashmi DT (USN0362), Bhoomika P (USN0090), Anita (USN0055) who are bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2026–27. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated into the project and in this report.

Signature of Guide	**Signature of Program Director**	**Signature of Dean**
Dr. J. Cynthia		**Dr. Shobha G**

External Viva:

Name of Examiners	
1	**Signature with Date**

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

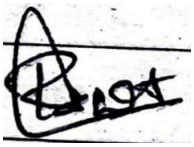
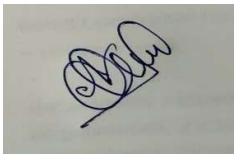
DECLARATION

We, Rashmi DT (USN0362), Bhoomika P (USN0090) and Anita (USN0055), students of fifth semester Minor in FinTech, School of Computer Science and Engineering, RV University, Bengaluru, hereby declare that the CS2614 Fundamentals of Blockchain mini project titled 'FRAUD DETECTION AND TRANSACTION MONITORING LAYER FOR ON-CHAIN FINANCE' has been carried out by us and submitted in partial fulfilment of the requirements of the course CS2614 Fundamentals of Blockchain during the year 2026–27.

Further we declare that the content of this report has not been submitted previously by anybody or to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RV University will be the property of RV University, Bengaluru and we will be the authors of the same.

Place: Bengaluru Date:

No.	Name	Signature
1	Rashmi DT (USN0362)	
2	Bhoomika P (USN0090)	
3	Anita (USN0055)	

ACKNOWLEDGEMENT

We express our sincere and heartfelt gratitude to Dr. J. Cynthia, project guide, for the constant guidance, valuable suggestions, technical expertise, and continuous encouragement provided throughout the course of our mini project work.

We are deeply grateful to Dr. D. Sathya, Dr. J. Cynthia and Mr. Rohith Raju, instructors of CS2614 Fundamentals of Blockchain, for the academic encouragement and support extended to us in successfully carrying out this project work. We take this opportunity to express our profound gratitude to Prof. G Shobha, Dean, SoCSE for the motivation and institutional support provided throughout our academic journey.

We also extend our sincere thanks to Prof. Sahana D. Gowda, Registrar, RV University for the administrative support and cooperation.

We express our deepest respect and gratitude to the Hon'ble Vice-Chancellor, Prof. Ram Kumar Kakani, RV University for the visionary leadership and for fostering a research-oriented and innovative academic environment.

Further, we extend our heartfelt thanks to the Management of RV University for providing excellent infrastructure, state-of-the-art facilities, and a conducive learning environment that enabled us to complete this project successfully.

We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the mini project work.

ABSTRACT:

Public blockchains let anyone create a wallet and move value within seconds, which makes it easy to hide where funds came from. Exchanges, wallet providers and DeFi protocols must monitor transfers for money-laundering patterns, but reviewing raw chain data by hand is slow, black-box scores are hard to justify, and ordinary alert logs can be edited later. A trustworthy, tamper-proof record of what was flagged and who decided is therefore needed.

Blockchain and smart contracts suit this problem because a contract can keep alerts and review decisions as permanent, time-stamped events that only authorised analysts can create. We built RiskRegistry, a Solidity 0.8.24 contract, using Hardhat and ethers, tested it on a local network and deployed it on the Sepolia testnet. Detection runs off-chain in a Python engine, since scanning every transfer on-chain would cost too much.

The system reads live Ethereum mainnet blocks through the Alchemy JSON-RPC service in read-only mode. No oracle feeds the contract. Approved alerts reach it through a queue file that a separate Hardhat script signs and sends to Sepolia, so the dashboard never holds a key.

The engine applies eight rules and an Isolation Forest and combines them into a 0–100 risk score with named reasons. An analyst dashboard groups alerts into cases and enforces a review workflow. The contract passes 15 test cases, eight of them negative, and the Python engine passes 54 tests. On simulated data with 42 injected fraud scenarios, all 42 were detected, with precision 0.996 using an exchange allowlist and 0.606 without it. One alert was anchored on Sepolia for 683,007 gas. The results come from simulated fraud, so they show that the pipeline works, not how it would perform on real laundering.

Keywords: blockchain, Ethereum, fraud detection, anti-money laundering, transaction monitoring, Isolation Forest, smart contract, audit trail

TABLE OF CONTENTS

Acknowledgement i
Abstract ii
Table of Contents iii
List of Figures iv
List of Tables v
List of Abbreviations vi

INTRODUCTION 1
OBJECTIVES 1
PROBLEM STATEMENT AND DOMAIN LINKAGE 2
LITERATURE REVIEW 2
EXISTING SYSTEM 4
PROPOSED SYSTEM 4
TECHNOLOGIES USED 4
ARCHITECTURE DIAGRAM 5
FEATURES 5
IMPLEMENTATION 6
RESULTS 9
SECURITY ANALYSIS AND LIMITATIONS 14
CONCLUSION 14
REFERENCES 16
APPENDIX 17

LIST OF FIGURES

Page No.

Figure 1: System architecture of the proposed system: off-chain detection, analyst review and on-chain audit trail 5

Figure 2: Contract deployment: Sepolia transaction with status Success, block 12053731, 6,426,712 gas used 11

Figure 3: Function execution: the raiseAlert transaction in block 12062466 (fee 0.000683 ETH) on the contract page 11

Figure 4: Events log: AlertRaised with alert ID 0, the flagged wallet, score 0x52 (82) and mask 0x10a (266), and the deployment events 12

Figure 5: Analyst dashboard in demo mode: 3,546 simulated transfers, 409 alerts, 45 cases and 1 case anchored on-chain 12

Figure 6: Case detail: why the wallet was flagged, with explanation, importance and next step for each signal, and the review actions 13

Figure 7: On-chain audit trail: alert #0 anchored, with transaction link and gas used (683,007) 13

Figure 8: Geo-tagged photograph of the batch during the mini project presentation 17

Figure 9: Hedera Part 1 – Student 1 18

Figure 10: Hedera Part 2 – Student 1 18

Figure 11: Hedera Part 1 – Student 2 18

Figure 12: Hedera Part 2 – Student 2 18

Figure 13: Hedera Part 1 – Student 3 18

Figure 14: Hedera Part 2 – Student 3 18

LIST OF TABLES

Page No.

Table 1: Linkage of the problem statement 2
Table 2: Details of the external source 2
Table 3: Literature review summary 3
Table 4: Existing systems 4
Table 5: Detection rules and weights 7
Table 6: Smart contract design 8
Table 7: Deployment details 8
Table 8: Test cases 9
Table 9: Gas analysis 10
Table 10: Detector evaluation on simulated data 13
Table 11: Team contribution 17
Table 12: Details of the geo-tagged photograph 17
Table 13: Project video and repository links 18
Table 14: Details of the conference paper draft 18
Table 15: Commands to run the prototype 19

LIST OF ABBREVIATIONS

Abbreviation	Definition
ABI	Application Binary Interface
AML	Anti-Money Laundering
API	Application Programming Interface
DApp	Decentralised Application
DeFi	Decentralised Finance
DLT	Distributed Ledger Technology
ERC	Ethereum Request for Comments
ETH	Ether (cryptocurrency of Ethereum)
EVM	Ethereum Virtual Machine
FATF	Financial Action Task Force
FinTech	Financial Technology
IPFS	InterPlanetary File System
KYC	Know Your Customer
ML	Machine Learning
PoS	Proof of Stake
PoW	Proof of Work
RPC	Remote Procedure Call
USN	University Serial Number

INTRODUCTION:

Ethereum lets anyone create a wallet without an identity check and send value within seconds. This openness is useful for finance, but it also makes it easy to hide where funds came from. Regulators extend anti-money-laundering (AML) duties to businesses that handle virtual assets, and the Financial Action Task Force (FATF) asks them to follow a risk-based approach and to monitor transactions [1]. The chain is public, so the data needed to spot suspicious behaviour is available. What is missing is a tool that turns this data into alerts an analyst can understand and defend, together with a record of those alerts that nobody can quietly change.

A blockchain is a distributed ledger that is copied across many nodes and agreed by consensus, so one party cannot alter or delete an entry after it is recorded. Ethereum adds smart contracts, which are programs written in Solidity that run on the Ethereum Virtual Machine and execute exactly as coded. For this domain, a smart contract is a good place to keep alerts and review decisions, because it can limit who is allowed to write, reject invalid or duplicate entries, and emit events that form a permanent, time-stamped trail.

In this project, detection runs off-chain because scanning every transfer on-chain would be far too expensive, and the chain is used only for the audit trail. A Python engine reads live Ethereum mainnet blocks, applies rules and an Isolation Forest, and gives each transfer an explained risk score. An analyst reviews the resulting cases on a dashboard, and approved alerts are anchored in the RiskRegistry contract on the Sepolia testnet. The scope covers live ingestion of native ETH transfers, scoring, the review workflow and testnet anchoring. It does not cover mainnet deployment or real funds, blocking or freezing transactions, ERC-20 and internal transfers, or identity (KYC) data.

Compliance teams at exchanges, custodians and wallet providers have to find and document suspicious activity, and auditors and regulators need evidence of what was flagged and who decided. A system that explains each alert and keeps a tamper-evident record of the decision speaks to both needs. The prototype shows how this can be done with open tools and a public testnet.

OBJECTIVES:

- To develop a monitoring layer that ingests live Ethereum mainnet transfers through JSON-RPC into a local database.

- To implement eight transparent AML rules and an Isolation Forest outlier detector, and to explain every alert with named reason codes.
- To test and deploy a role-based RiskRegistry smart contract, written in Solidity, on a local Hardhat network and on the Sepolia testnet.
- To demonstrate an analyst dashboard for reviewing cases and anchoring alerts on-chain, and to evaluate the detector on simulated fraud.

PROBLEM STATEMENT AND DOMAIN LINKAGE:

Problem Statement: Exchanges, wallet providers and DeFi protocols need to detect suspicious activity on a public chain in near real time, but manual review of raw transaction data is slow, black-box scores cannot be justified to an auditor, and pure rule engines are easy to evade. Alert logs and review decisions also usually sit in an ordinary database, where nothing proves that an alert was not deleted or a decision changed later.

Linkage of the Problem Statement:

Table 1: Linkage of the problem statement

Linked to	Team's specification
FinTech	AML transaction monitoring for crypto payments and DeFi. The affected process is the detection, escalation and documentation of suspicious activity by the compliance analyst. The stakeholders are compliance officers at exchanges, custodians and wallet providers, DeFi protocols and their users, and auditors and regulators.
Blockchain	Blockchain is used for the immutability and transparency of the audit trail. The contract keeps each alert and every review decision as a permanent, time-stamped event that only authorised analysts can create and nobody can delete. Access control, duplicate protection and the review state rules run on-chain in the smart contract.
External source (live data / off-chain data)	Live Ethereum mainnet blocks through the Alchemy JSON-RPC service, read-only, about every 12 seconds. Detection runs off-chain. Approved alerts reach the contract through a queue file that a separate script signs on the Sepolia testnet.

Details of the External Source:

Table 2: Details of the external source

Item	Details
Name and URL of the source	Ethereum mainnet JSON-RPC endpoint provided by Alchemy (alchemy.com)
Type of data	Live data
How the data reaches the contract	It does not reach the contract directly. The Python ingestion module reads blocks over https JSON-RPC and stores them in SQLite. Only analyst-approved alerts are sent to the contract, by the anchoring script.
Update frequency	About every 12 seconds (one Ethereum block)

Item	Details
Trust and security consideration	The connection is read-only, uses https only, redacts the API key from logs and retries on network errors. Wrong or delayed data can cause a late, missed or false alert, but no alert is anchored without analyst review.
Where it is used	The RPC client and ingestion module of the Python monitor package; the contract itself reads no external data.

LITERATURE REVIEW:

Five sources shaped the design. Weber et al. [2] trained graph convolutional networks on a labelled Bitcoin transaction graph and showed that graph structure helps to find illicit transactions, but the work is an offline experiment on a single labelled dataset. Chen et al. [3] detected phishing accounts on Ethereum by extracting graph-based cascade features and training a LightGBM ensemble; the method needs labelled phishing accounts and classifies accounts instead of producing explained, recorded alerts. Hassan, Rehmani and Chen [4] survey anomaly detection in blockchain networks, covering evaluation metrics, requirements and models by blockchain layer, and list open challenges, but a survey does not give a working monitoring layer. Luo et al. [5] organise DeFi fraud by project life-cycle stage and review statistical, machine-learning and graph methods; they note that detection at early stages is immature because data and reliable ground truth are scarce. Liu, Ting and Zhou [6] introduced Isolation Forest, an unsupervised method that isolates outliers with random trees and needs no labelled fraud, which suits a setting where labels are rare.

Table 3: Literature review summary

Sl.	Author(s) & Year	Title / Source	Approach	Findings	Limitation / Gap
1	Weber et al., 2019 [2]	Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics – KDD Workshop on Anomaly Detection in Finance	Graph convolutional network on a labelled Bitcoin graph	Graph structure helps to find illicit transactions	Offline; one dataset; no alerting workflow or audit trail
2	Chen et al., 2020 [3]	Phishing scam detection on Ethereum: Towards financial security for blockchain ecosystem – IJCAI 2020	Graph cascade features with a LightGBM ensemble	Detects phishing accounts on Ethereum	Supervised; needs labelled accounts; no explained, anchored alerts

Sl.	Author(s) & Year	Title / Source	Approach	Findings	Limitation / Gap
3	Hassan et al., 2022 [4]	Anomaly detection in blockchain networks: A comprehensive survey – arXiv:2112.06089	Survey of anomaly detection by blockchain layer	Covers metrics, requirements and models; lists open challenges	Survey only; no working monitoring layer
4	Luo et al., 2023 [5]	AI-powered fraud detection in decentralized finance: A project life cycle perspective – arXiv:2308.15992	Taxonomy of DeFi fraud and detection methods by life-cycle stage	Early-stage detection is immature because ground truth is scarce	A review, not a tool
5	Liu, Ting and Zhou, 2008 [6]	Isolation forest – IEEE ICDM 2008	Unsupervised outlier detection with random trees	Isolates outliers without labelled fraud	General method; needs AML features, rules and an explanation layer

Research gap: Existing work either detects well offline or surveys the field. None of the sources combines real-time monitoring, scores that break down into reasons an analyst can read, and a tamper-evident record of the analyst's decision. This project targets that combination.

EXISTING SYSTEM:

Table 4: Existing systems

Sl.	Existing system / method	How it works	Strengths	Limitations
1	Manual review of chain data	Analysts read raw public transaction data and look for suspicious patterns by hand	Human judgement; no model to maintain	Slow on large data; patterns such as splitting funds across many wallets are easy to miss
2	Black-box ML scoring	A trained model gives each wallet or transfer a risk number	Can find patterns that no one wrote down as a rule	Gives a number without a reason, which is hard to defend to an auditor
3	Rule-only engine with database logs	Fixed rules raise alerts, and the alerts and decisions are stored in an ordinary database	Transparent and easy to explain	Easy to evade; records can be edited, so nothing proves an alert was not deleted or a decision changed

PROPOSED SYSTEM:

The proposed system splits the work between an off-chain engine and an on-chain contract. A Python engine ingests live Ethereum transfers, applies eight rules and an Isolation Forest, and

combines the signals into a 0–100 risk score in which every point can be traced to a named reason. Alerts are grouped into one case per wallet on an analyst dashboard, where the analyst marks a case Reviewed, Escalated or Dismissed. Approved alerts are anchored in the RiskRegistry contract, which stores only a transaction hash, the flagged wallet, the score, a reason bitmask and a review status.

There are three actors. The analyst reviews cases and anchors alerts, the owner manages the list of analysts and the on-chain watchlist, and the off-chain engine produces the alerts. Compared with the existing systems, the explained score replaces black-box output, the outlier detector is added to the rules so that evasion of fixed thresholds is harder, and the contract turns every alert and decision into a permanent event that cannot be deleted.

TECHNOLOGIES USED:

- **Smart Contract Language:** Solidity 0.8.24 [7]
- **Development Environment:** Hardhat, VS Code, Node.js; Python for the detection engine
- **Local Blockchain:** Hardhat local network (used for the contract and anchoring tests)
- **Wallet:** Throwaway testnet deployer account whose key is held only in the local .env file
- **Frontend / Library:** ethers (anchoring script); HTML, CSS and JavaScript dashboard served by a Python standard-library HTTP server
- **Detection Libraries:** pandas, NumPy, scikit-learn; SQLite (WAL mode) for storage
- **Test Network:** Sepolia
- **External Data:** Alchemy JSON-RPC service for Ethereum mainnet blocks (no price-feed oracle is used)
- **Version Control:** GitHub – https://github.com/rashmideepaktoragallamath/onchain-fraud-monitor

ARCHITECTURE DIAGRAM:

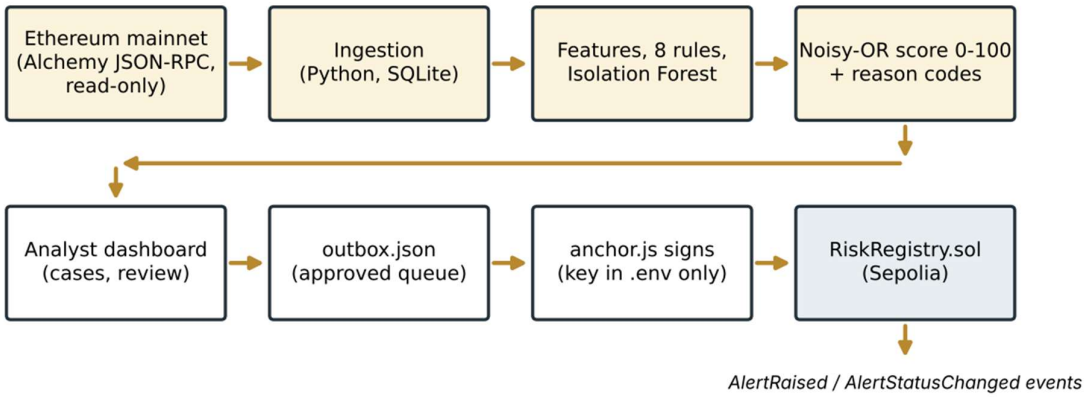


Figure 1: System architecture of the proposed system: off-chain detection, analyst review and on-chain audit trail

- **Data source:** Ethereum mainnet through the Alchemy JSON-RPC service. Blocks are read, never written.
- **Ingestion:** Python and SQLite. It fetches new blocks, stores each ETH transfer, keeps a cursor and retries on errors.
- **Detection:** pandas, NumPy and scikit-learn compute features, apply the eight rules and the Isolation Forest, and produce the noisy-OR score.
- **Dashboard / API:** A Python HTTP server bound to localhost shows cases and charts, runs the review workflow and writes the anchoring queue. It holds no key.
- **Anchoring:** A Node.js and Hardhat script, scripts/anchor.js, is the only component that signs transactions, using the deployer key from .env.
- **Smart contract:** RiskRegistry on Sepolia keeps the immutable alert and review record.

FEATURES:

- **Live Transaction Ingestion:** New mainnet blocks are read about every 12 seconds, and a cursor lets a restart continue where it stopped.
- **Explainable Risk Scoring:** Eight rules and an Isolation Forest are combined with a noisy-OR formula. Each alert lists its reason codes, a plain-language explanation and a suggested next step.
- **Case Management and Review Workflow:** Alerts are grouped into one case per wallet, and the allowed status changes match the contract.

- **On-Chain Anchoring:** An approved alert is stored with its transaction hash, flagged wallet, score, reason mask and status. A transaction can be reported only once.
- **Access Control:** The onlyOwner and onlyAnalyst modifiers separate who manages roles and the watchlist from who writes alerts, and ownership moves in two steps.
- **Event Logging:** AlertRaised, AlertStatusChanged, WatchlistUpdated, AnalystUpdated and OwnershipTransferred events give a permanent trail.
- **External Data Integration:** Live Ethereum mainnet data is read through JSON-RPC; the contract itself uses no external data.

IMPLEMENTATION:

1. **Requirement Analysis & Design:** The problem, the actors (analyst, owner and off-chain engine) and the use cases were identified. It was decided to keep detection off-chain and the audit record on-chain.
2. **Smart Contract Development (Solidity):** RiskRegistry was written with alert records, mappings, modifiers, events and custom errors, and a two-step ownership transfer. It uses no inheritance or oracle.
3. **Compilation:** The contract was compiled with Hardhat using Solidity 0.8.24.
4. **Local Deployment and Testing:** The contract was deployed on the Hardhat local network and tested with 15 test cases. The anchoring logic was tested separately with 8 tests against a fake contract.
5. **Test Network Deployment and Integration:** The contract was deployed on Sepolia, and the anchoring script uses ethers to send transactions to it.
6. **Off-Chain Engine and Dashboard:** The Python engine, simulator, evaluation module and dashboard were built and tested with 54 tests.

Off-Chain Detection Engine:

The ingestion module calls the JSON-RPC methods of an Ethereum node through Alchemy to read new blocks. It stores each native ETH transfer (hash, block, time, sender, receiver, value and nonce) in a SQLite database. For repeatable testing, a simulator generates a background of normal transfers and injects labelled fraud scenarios. For each transfer the engine computes behavioural features over a one-hour sliding window, and eight rules map to known AML typologies, as listed below. The monitor package has about 2,100 lines of Python, and a heuristic marks wallets that look like exchanges or bots, because their high-volume behaviour resembles layering.

Table 5: Detection rules and weights

Rule (reason code)	Condition used by the prototype	Weight
Structuring	3 or more transfers of 9–10 ETH, just under the 10 ETH review limit	0.55
Fan-out	One wallet sends to 15 or more distinct recipients within the window	0.50
Fan-in	15 or more wallets send to one wallet within the window	0.50
Round-trip	Funds return to their origin within ±25% of the amount	0.50
High velocity	20 or more transfers by one sender within the window	0.50
Watchlist hit	Counterparty is on the watchlist file (data/watchlist.json)	0.75
Large value	Transfer of 100 ETH or more	0.35
New-wallet burst	Wallet with nonce 5 or lower moves 20 ETH or more	0.35
ML outlier	Isolation Forest flags the feature vector (top 1% most isolated)	0.30

The thresholds are configurable and were chosen by the team for the prototype. They are not calibrated against real compliance data.

Risk score. Each signal i has a weight w_i between 0 and 1, and independent signals are combined with a noisy-OR formula: score = 100 × (1 − Π(1 − w_i)). The score never exceeds 100 and never decreases when more evidence appears. A single weak signal stays below the alert threshold of 50, so context signals (large value, young wallet, ML outlier) need corroboration, while pattern rules and watchlist hits can raise an alert on their own. The severity bands are critical for 80 or more, high for 65–79 and medium for 50–64. For example, high velocity (0.50), fan-out (0.50) and ML outlier (0.30) give 100 × (1 − 0.5 × 0.5 × 0.7) = 82.5, shown as 82. Because the score is a product of named terms, it can always be broken down into the reasons that produced it.

Cases and review workflow. One wallet can raise many alerts, so the dashboard groups them into one case per wallet; for fan-in alerts the case belongs to the receiver. An analyst can mark a case Reviewed, Escalated or Dismissed. The rules match the contract: Open may move to any other state, Reviewed may move to Escalated or Dismissed, and Escalated and Dismissed are final. A known exchange can be allowlisted, which dismisses its case and suppresses future behaviour-based alerts for it.

On-chain anchoring. The dashboard never signs transactions. When an analyst queues a case, it writes data/outbox.json. A separate Hardhat script (npm run anchor) reads the queue, signs with the deployer key held only in the local .env file, calls raiseAlert and updateStatus on RiskRegistry, and writes the alert ID, transaction hash and gas used to data/anchored.json, which the dashboard merges. The script is idempotent: a second run sends no duplicate

transactions, never attempts a transition the contract would reject, and sends at most 10 transactions per run.

Dashboard. The dashboard (python -m monitor.serve) has three modes: --demo (simulated fraud with its own database), --live (real mainnet blocks refreshed every 12 seconds) and a viewer for the stored live database. It shows summary tiles, alerts over time, active cases by severity, the most common signals, a filterable case list and a case detail panel organised as Detect, Explain and Prove.

Smart Contract Design:

Table 6: Smart contract design

Component	Type	Purpose
owner, pendingOwner	State variable	The owner manages analysts and the watchlist; ownership moves in two steps (propose, accept)
isAnalyst	State mapping	Only approved analysts can write alerts or change status
_alerts, _alertIdPlusOne	State variable	Alert records, and a map from transaction hash to alert so a transaction is reported only once
raiseAlert()	Function	Called by an analyst. Rejects an empty hash, zero address, score of 0 or above 100, empty reason mask and duplicate transactions
updateStatus()	Function	Called by an analyst. Open to Reviewed/Escalated/Dismissed; Reviewed to Escalated/Dismissed; final states cannot change
addToWatchlist() / removeFromWatchlist()	Function	Owner only; a reason is required
AlertRaised, AlertStatusChanged, WatchlistUpdated, AnalystUpdated, OwnershipTransferred	Event	Emitted on every action, giving a permanent audit trail
onlyOwner, onlyAnalyst	Modifier	Role-based access control; custom errors are used to save gas

Reason bitmask. Each reason has one bit: large value (bit 0), high velocity (1), structuring (2), fan-out (3), fan-in (4), round-trip (5), watchlist (6), new-wallet burst (7) and ML outlier (8). The Python engine and the contract use the same bit positions. The mask 266 in the demo alert equals 256 + 8 + 2, which means ML outlier, fan-out and high velocity. Only a transaction hash, a public wallet address, a score from 1 to 100 and a 16-bit reason mask are stored on-chain, so no personal data is kept.

Key Code Snippet:

```
contract RiskRegistry {
```

```
/*//////////////////////////////////////////////////////////////
                            TYPES
    //////////////////////////////////////////////////////////////*/
enum Status {
   Open,
        Reviewed,
        Escalated,
        Dismissed
    }
struct Alert {
bytes32 txHash; // hash of the monitored transaction
address subject; // wallet the alert is about
 address raisedBy; // analyst who anchored the alert
 uint64 raisedAt; // block timestamp
        uint64 updatedAt; // last status change
        uint16 reasonMask; // bit i set => reason code i triggered
       uint8 riskScore; // 1..100
       Status status;
}
/*//////////////////////////////////////////////////////////////
STORAGE
```

```
    //////////////////////////////////////////////////////////////*/
    uint8 public constant MAX_RISK_SCORE = 100;


    address public owner;
    address public pendingOwner;
    mapping(address => bool) public isAnalyst;
    mapping(address => bytes32) public watchlistReason; // non-zero => watchlisted
Alert[] private _alerts;
    mapping(bytes32 => uint256) private _alertIdPlusOne; // txHash => id + 1
    /*//////////////////////////////////////////////////////////////
                                EVENTS
    //////////////////////////////////////////////////////////////*/
event AlertRaised(
        uint256 indexed alertId,
        bytes32 indexed txHash,
        address indexed subject,
        uint8 riskScore,
        uint16 reasonMask
    );
event AlertStatusChanged(uint256 indexed alertId, Status from, Status to, address indexed
by);
    event AnalystUpdated(address indexed analyst, bool enabled);
    event WatchlistUpdated(address indexed account, bool listed, bytes32 reason);
    event OwnershipTransferStarted(address indexed previousOwner, address indexed
newOwner);
    event OwnershipTransferred(address indexed previousOwner, address indexed
newOwner);
```

Deployment Details:

Table 7: Deployment details

Item	Details
Network	Hardhat local network (tests); Sepolia testnet (deployment)
Contract address	0x1D5a2F95057EFbbb596b9E14A7dAD3975190924b
Deployment transaction hash	0x4a65b32c91ba7a2513f198bb89849cdfeb8fa5b3f20597b2b9a89d94d334d4df
Deployment block	12053731
Compiler version	Solidity 0.8.24
GitHub repository	https://github.com/rashmideepaktoragallamath/onchain-fraud-monitor

RESULTS:

- The contract compiled and deployed successfully on the local network and on Sepolia (block 12053731, 6,426,712 gas).
- The main functionality works as intended. All 15 contract tests pass, along with 8 anchoring tests and 54 Python tests. Alert #0 was anchored on Sepolia with risk score 82 and reason mask 266.
- Access control and invalid inputs are handled correctly: calls by non-analysts and non-owners, duplicate transactions, invalid scores and changes to a final state all revert.
- Live mainnet data was fetched and used correctly. One live run ingested 2,365 real transfers and raised 665 alerts.
- Gas used was 6,426,712 for deployment and 683,007 for the raiseAlert transaction (fee 0.000683 ETH at about 1 Gwei).

Test Cases:

Table 8: Test cases

TC ID	Test scenario	Input	Expected result	Actual result	Status
TC-01	Deployment	Deploy RiskRegistry	Deployer is owner and analyst	As expected	Pass
TC-02	Analyst anchors an alert	raiseAlert() from an analyst with valid data	AlertRaised emitted, alert stored	As expected	Pass
TC-03	Look up alert by transaction hash	Hash of an anchored transaction	Correct alert ID returned	As expected	Pass
TC-04	Negative: non-analyst raises alert	raiseAlert() from a non-analyst	Revert	Reverted	Pass
TC-05	Negative: same transaction reported twice	Same transaction hash twice	Revert AlreadyReported	Reverted	Pass
TC-06	Negative: invalid alert data	Score 0 or above 100, empty reasons, zero hash, zero address	Revert	Reverted	Pass

TC ID	Test scenario	Input	Expected result	Actual result	Status
TC-07	Open to Reviewed to Escalated	Two updateStatus() calls	Two AlertStatusChanged events	As expected	Pass
TC-08	Negative: change a final state	updateStatus() on a final alert	Revert	Reverted	Pass
TC-09	Negative: move back to Open or stay in same state	Invalid status change	Revert	Reverted	Pass
TC-10	Negative: outsider updates status; unknown alert ID	updateStatus() by a non-analyst; unknown ID	Revert	Reverted	Pass
TC-11	Owner adds and removes a watchlist address	addToWatchlist() and removeFromWatchlist()	WatchlistUpdated events	As expected	Pass
TC-12	Negative: non-owner edits watchlist; empty reason	Watchlist call by a non-owner; empty reason	Revert	Reverted	Pass
TC-13	Revoked analyst loses write access	raiseAlert() after the analyst is revoked	Revert after revoke	Reverted	Pass
TC-14	Two-step ownership transfer	Propose, then accept	Owner changes only after acceptance	As expected	Pass
TC-15	Negative: non-owner manages analysts; zero address	Analyst update by a non-owner; zero address	Revert	Reverted	Pass

All 15 tests pass, and 8 of them are negative tests that expect a revert. One test failure found while testing on Windows led to a real fix: the server now reads and discards a bounded amount of an unread request body before it sends an error, so a rejected request no longer resets the connection.

Gas Analysis:

Table 9: Gas analysis

Function	Gas used	Remarks
Deployment	6,426,712	One-time cost on Sepolia
raiseAlert() (alert #0)	683,007	Fee 0.000683 ETH at about 1 Gwei

These figures come from one run on a testnet. Gas depends mainly on storage writes and was not tuned in this project.

Deployment and Transaction Evidence:

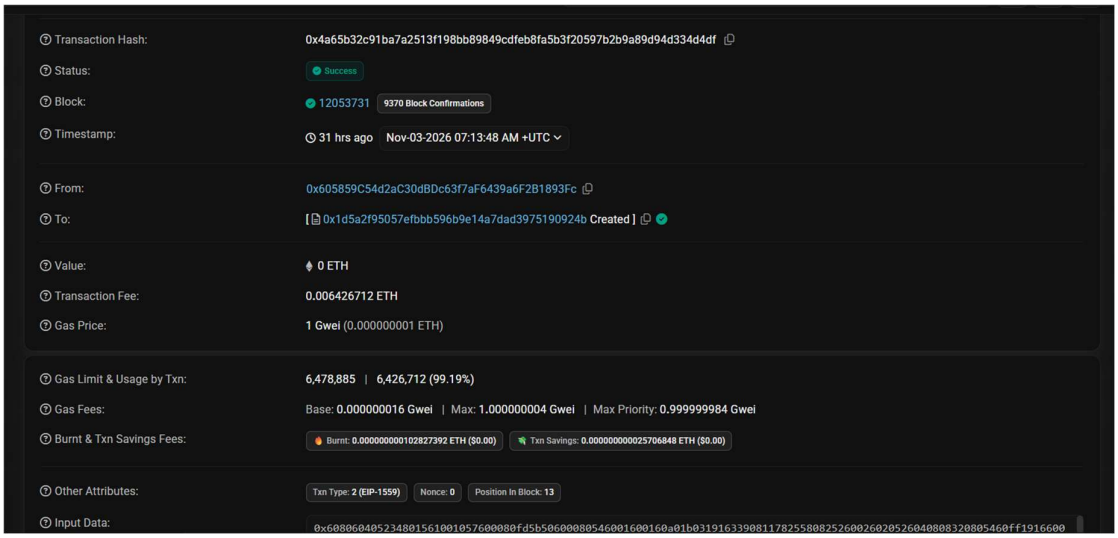


Figure 2: Contract deployment: Sepolia transaction with status Success, block 12053731, 6,426,712 gas used

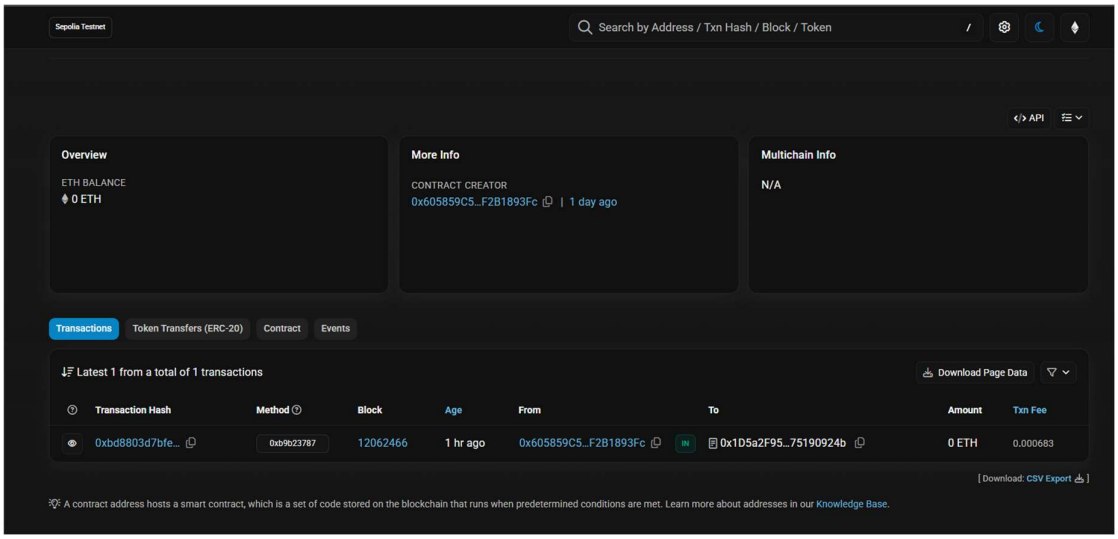


Figure 3: Function execution: the raiseAlert transaction in block 12062466 (fee 0.000683 ETH) on the contract page

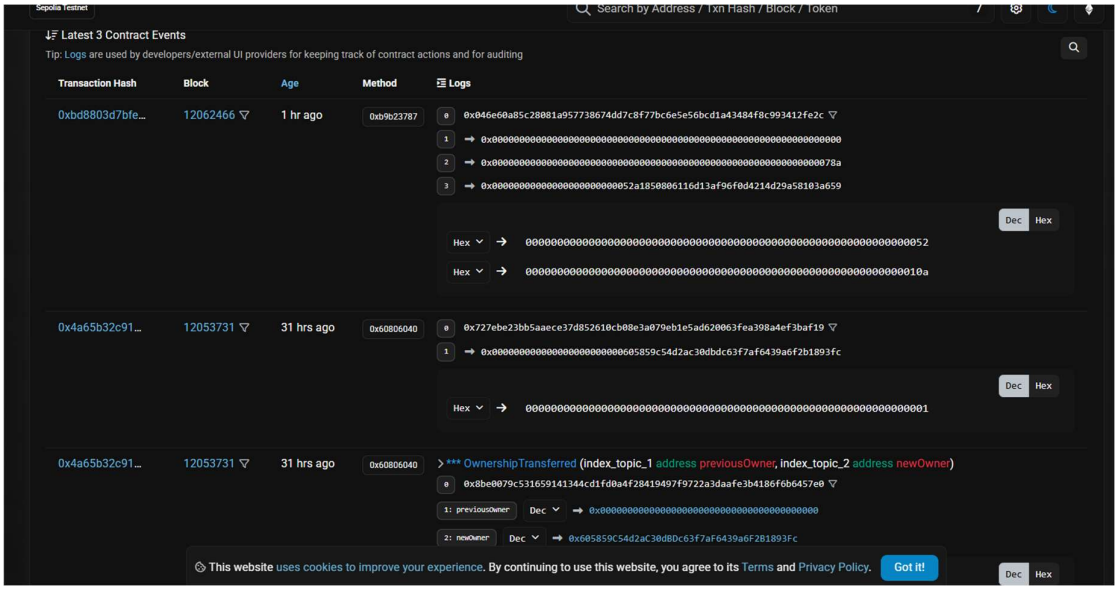


Figure 4: Events log: AlertRaised with alert ID 0, the flagged wallet, score 0x52 (82) and mask 0x10a (266), and the deployment events

The transaction hash anchored in the demo belongs to a simulated transfer, because the demo runs on simulated data. The Sepolia transactions themselves are real.

Dashboard Output:

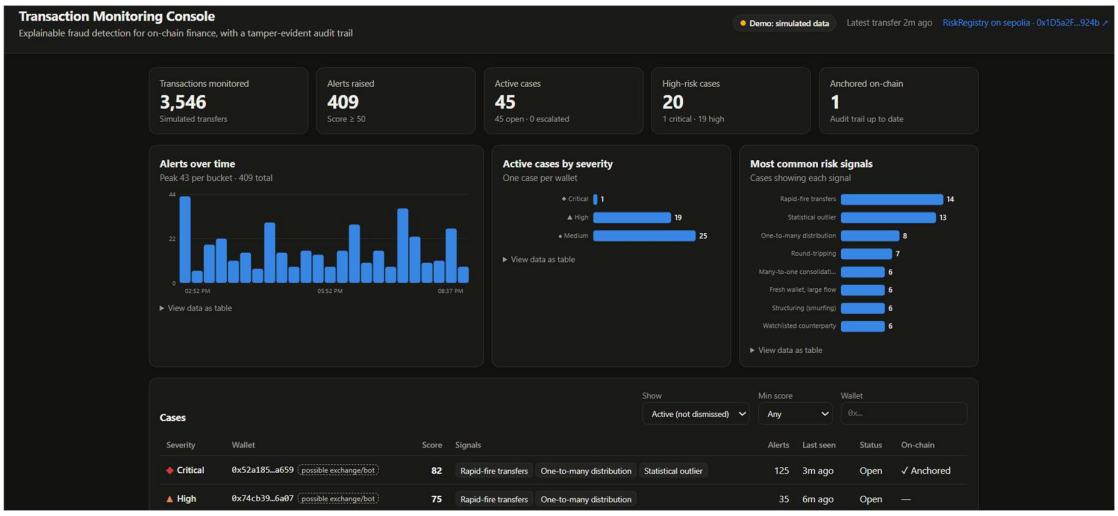


Figure 5: Analyst dashboard in demo mode: 3,546 simulated transfers, 409 alerts, 45 cases and 1 case anchored on-chain

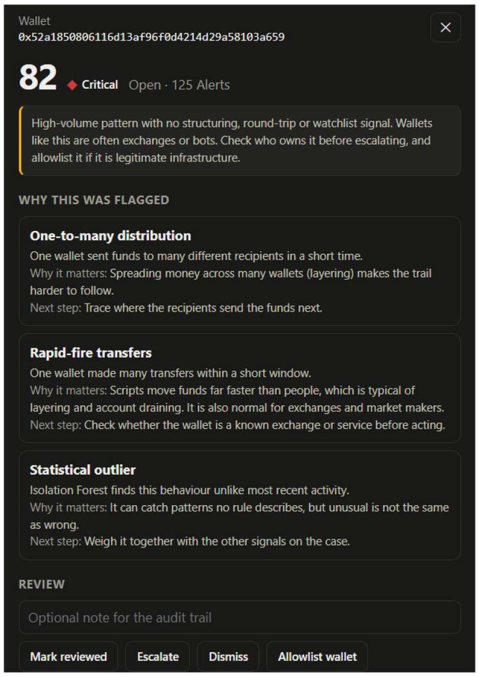


Figure 6: Case detail: why the wallet was flagged, with explanation, importance and next step for each signal, and the review actions

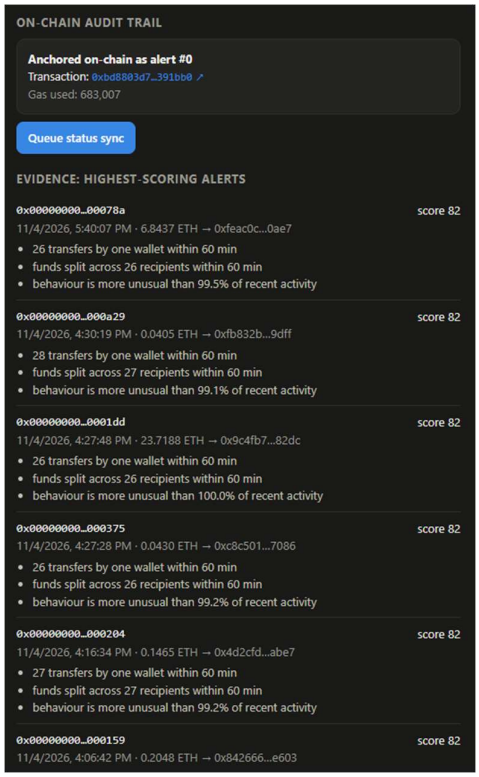


Figure 7: On-chain audit trail: alert #0 anchored, with transaction link and gas used (683,007)

Detector Evaluation on Simulated Fraud:

The simulator produced 3,546 transfers, of which 552 belong to 42 injected fraud scenarios (six for each of seven patterns: structuring, fan-out, fan-in, round-trip, watchlist, new-wallet burst and high velocity). The alert threshold was 50 and the random seed was 7.

Table 10: Detector evaluation on simulated data

Measure	With exchange allowlist	Without allowlist
Alerts	249	409
Alerts inside fraud scenarios	248	248
False-positive alerts	1	161
False-positive rate (of normal transfers)	0.03%	5.38%
Alert precision	0.996	0.606
Scenarios detected	42 of 42	42 of 42
Scenarios with the expected reason code	42 of 42	42 of 42

All 42 scenarios were found, and each was explained with the expected reason. Without the allowlist, the false alerts come from legitimate high-volume wallets such as exchanges and routers that look like layering, which is why the allowlist and human review are part of the design. These results need to be read with care. The fraud scenarios were written by the same team that designed the rules, and the allowlist contains the simulator's own exchange addresses.

The numbers therefore show that the pipeline works as designed, but not how accurate it would be against real laundering.

Live Mainnet Run:

In live mode, one run ingested 2,365 real transfers and raised 665 alerts. There is no ground truth for mainnet data, so no accuracy is claimed. The dashboard labels high-volume wallets that look like exchanges or bots as "possible exchange/bot", and its guidance tells the analyst to check ownership before escalating. This confirms that real traffic produces false positives that an analyst has to resolve.

SECURITY ANALYSIS AND LIMITATIONS:

The contract uses role-based access (onlyOwner and onlyAnalyst), validates its inputs with custom errors, rejects duplicate transactions, makes Escalated and Dismissed final and moves ownership in two steps. It stores no personal data. Around the contract, secrets are kept only in a .env file that is excluded from git, and a throwaway testnet wallet is used. The mainnet connection is read-only, so no mainnet key or funds exist anywhere in the system. The dashboard server is bound to localhost, checks Host and Origin headers, requires a custom header on writes, applies a strict Content-Security-Policy, limits request size, sanitises notes and renders wallet data as text only so it cannot inject script. The network client uses https only and redacts the API key from error messages. The anchoring script validates every queue item, never runs a transition the contract would revert, and caps transactions per run.

The only external data is the mainnet block feed. If it is delayed or incomplete, alerts may be late or missed, and real traffic also produces false positives, as the live run showed. These risks are reduced by keeping the feed read-only, retrying on network errors, labelling wallets that look like exchanges or bots, and requiring an analyst to review each case before anything is anchored. The contract never receives the feed directly, only analyst-approved alerts.

The project has several limitations. The evaluation uses simulated fraud designed alongside the rules, so high recall is expected and real laundering is more varied. The thresholds are hand-chosen and uncalibrated, and the mainnet run has no labels. Only native ETH transfers are monitored, not ERC-20 tokens, internal transfers or cross-chain flows. Exchanges raise false positives, and the allowlist is a manual list. Anchoring is run by an analyst through a script, so the audit trail depends on a single signing key, and a multi-signature scheme would be stronger.

Gas was not optimised, and the contract runs only on a testnet with no formal audit or mainnet deployment.

CONCLUSION:

The project delivers a working monitoring layer: live mainnet ingestion, eight rules plus an Isolation Forest, an explained 0–100 risk score, an analyst dashboard with a review workflow that mirrors the contract, and a deployed RiskRegistry that holds a tamper-evident record of alerts and decisions. All four objectives were achieved: live data was ingested (2,365 transfers), all injected scenarios were detected and explained, 15 of 15 contract tests passed with 8 negative cases, and alert #0 was anchored on Sepolia.

Testing showed that the contract rejects unauthorised and invalid actions, and that the engine finds every injected scenario on simulated data. Live data showed that exchange traffic causes false alarms, which supports keeping a human in the loop. The project demonstrates that blockchain fits the part of FinTech compliance where a record must be trusted, while the heavy analysis is better done off-chain.

Overall, the work shows how an explainable detector and a small, role-controlled smart contract can be combined into a practical audit trail. For the team, it was a hands-on exercise in Solidity, testing, testnet deployment and connecting off-chain systems to a contract.

Future Scope:

- Monitor ERC-20 and internal transfers, and add graph features and labelled data for supervised learning, with calibrated thresholds.
- Reduce gas with batched anchoring, and deploy on a low-cost layer-2 network.
- Require multi-signature approval for anchoring instead of a single signing key.
- Send alerts to compliance tools by webhook, and carry out a formal security audit.

REFERENCES:

[1] Financial Action Task Force, "Updated guidance for a risk-based approach to virtual assets and virtual asset service providers," Oct. 2021. [Online]. Available: https://www.fatf-gafi.org

[2] M. Weber et al., "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," KDD Workshop on Anomaly Detection in Finance, 2019, arXiv:1908.02591.

[3] W. Chen, X. Guo, Z. Chen, Z. Zheng, and Y. Lu, "Phishing scam detection on Ethereum: Towards financial security for blockchain ecosystem," in Proc. 29th Int. Joint Conf. Artificial Intelligence (IJCAI), 2020, pp. 4506–4512.

[4] M. U. Hassan, M. H. Rehmani, and J. Chen, "Anomaly detection in blockchain networks: A comprehensive survey," arXiv:2112.06089, 2022.

[5] B. Luo, Z. Zhang, Q. Wang, A. Ke, S. Lu, and B. He, "AI-powered fraud detection in decentralized finance: A project life cycle perspective," arXiv:2308.15992, 2023.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[7] Ethereum Foundation, "Solidity documentation." [Online]. Available: https://docs.soliditylang.org

APPENDIX:

A. Complete Solidity Source Code

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;


/// @title RiskRegistry

/// @notice Tamper-evident audit trail for a transaction-monitoring system.

///         Off-chain detection engines score transactions; authorised analysts

///         anchor the resulting alerts here so that neither the alert nor its

///         review history can be silently altered or deleted afterwards.

/// @dev    Personal data never goes on-chain: only hashes, addresses that are

///         already public, a 0-100 score and a bitmask of reason codes.

contract RiskRegistry {

    /*//////////////////////////////////////////////////////////////

                              TYPES

    //////////////////////////////////////////////////////////////*/


    enum Status {

        Open,

        Reviewed,

        Escalated,

        Dismissed

    }


    struct Alert {

        bytes32 txHash; // hash of the monitored transaction
```

```
    address subject; // wallet the alert is about
    address raisedBy; // analyst who anchored the alert
    uint64 raisedAt; // block timestamp
    uint64 updatedAt; // last status change
    uint16 reasonMask; // bit i set => reason code i triggered
    uint8 riskScore; // 1..100
    Status status;
}

/*//////////////////////////////////////////////////////////////
                        STORAGE
//////////////////////////////////////////////////////////////*/

uint8 public constant MAX_RISK_SCORE = 100;

address public owner;
address public pendingOwner;

mapping(address => bool) public isAnalyst;
mapping(address => bytes32) public watchlistReason; // non-zero => watchlisted

Alert[] private _alerts;
mapping(bytes32 => uint256) private _alertIdPlusOne; // txHash => id + 1

/*//////////////////////////////////////////////////////////////
                        EVENTS
```

```
    ////////////////////////////////////////////////////////////*/

    event AlertRaised(
        uint256 indexed alertId,
        bytes32 indexed txHash,
        address indexed subject,
        uint8 riskScore,
        uint16 reasonMask
    );
    event AlertStatusChanged(uint256 indexed alertId, Status from, Status to, address indexed
by);
    event AnalystUpdated(address indexed analyst, bool enabled);
    event WatchlistUpdated(address indexed account, bool listed, bytes32 reason);
    event OwnershipTransferStarted(address indexed previousOwner, address indexed
newOwner);
    event OwnershipTransferred(address indexed previousOwner, address indexed
newOwner);

    /*////////////////////////////////////////////////////////////
                            ERRORS
    ////////////////////////////////////////////////////////////*/

    error NotOwner();
    error NotPendingOwner();
    error NotAnalyst();
    error ZeroAddress();
    error EmptyTxHash();
```

```
error InvalidRiskScore(uint8 score);
error NoReasonGiven();
error AlreadyReported(bytes32 txHash, uint256 alertId);
error UnknownAlert(uint256 alertId);
error InvalidTransition(Status from, Status to);
error EmptyReason();

/*//////////////////////////////////////////////////////////////
                        MODIFIERS
//////////////////////////////////////////////////////////////*/

modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

modifier onlyAnalyst() {
    if (!isAnalyst[msg.sender]) revert NotAnalyst();
    _;
}

/*//////////////////////////////////////////////////////////////
                      CONSTRUCTOR
//////////////////////////////////////////////////////////////*/

constructor() {
```

```
        owner = msg.sender;
        isAnalyst[msg.sender] = true;
        emit OwnershipTransferred(address(0), msg.sender);
        emit AnalystUpdated(msg.sender, true);
    }

    /*//////////////////////////////////////////////////////////////
                            ALERT LIFECYCLE
    //////////////////////////////////////////////////////////////*/

    /// @notice Anchor a new alert. A transaction can be reported only once.
    function raiseAlert(bytes32 txHash, address subject, uint8 riskScore, uint16 reasonMask)
        external
        onlyAnalyst
        returns (uint256 alertId)
    {
        if (txHash == bytes32(0)) revert EmptyTxHash();
        if (subject == address(0)) revert ZeroAddress();
        if (riskScore == 0 || riskScore > MAX_RISK_SCORE) revert
InvalidRiskScore(riskScore);
        if (reasonMask == 0) revert NoReasonGiven();

        uint256 existing = _alertIdPlusOne[txHash];
        if (existing != 0) revert AlreadyReported(txHash, existing - 1);

        alertId = _alerts.length;
```

```
        _alerts.push(
            Alert({
                txHash: txHash,
                subject: subject,
                raisedBy: msg.sender,
                raisedAt: uint64(block.timestamp),
                updatedAt: uint64(block.timestamp),
                reasonMask: reasonMask,
                riskScore: riskScore,
                status: Status.Open
            })
        );
        _alertIdPlusOne[txHash] = alertId + 1;

        emit AlertRaised(alertId, txHash, subject, riskScore, reasonMask);
    }

    /// @notice Move an alert through review. Escalated and Dismissed are final.
    ///         Allowed: Open -> Reviewed | Escalated | Dismissed,
    ///                  Reviewed -> Escalated | Dismissed.
    function updateStatus(uint256 alertId, Status newStatus) external onlyAnalyst {
        if (alertId >= _alerts.length) revert UnknownAlert(alertId);
        Alert storage a = _alerts[alertId];
        Status current = a.status;

        bool allowed = (current == Status.Open && newStatus != Status.Open)
```

```
        || (current == Status.Reviewed && (newStatus == Status.Escalated || newStatus ==
Status.Dismissed));
        if (!allowed) revert InvalidTransition(current, newStatus);

        a.status = newStatus;
        a.updatedAt = uint64(block.timestamp);
        emit AlertStatusChanged(alertId, current, newStatus, msg.sender);
    }

    /*//////////////////////////////////////////////////////////////
                            WATCHLIST
    //////////////////////////////////////////////////////////////*/

    /// @notice Flag an address (e.g. sanctioned or known-bad). `reason` must be non-zero.
    function addToWatchlist(address account, bytes32 reason) external onlyOwner {
        if (account == address(0)) revert ZeroAddress();
        if (reason == bytes32(0)) revert EmptyReason();
        watchlistReason[account] = reason;
        emit WatchlistUpdated(account, true, reason);
    }

    function removeFromWatchlist(address account) external onlyOwner {
        delete watchlistReason[account];
        emit WatchlistUpdated(account, false, bytes32(0));
    }
```

```
function isWatchlisted(address account) external view returns (bool) {
    return watchlistReason[account] != bytes32(0);
}

/*//////////////////////////////////////////////////////////////
                        ACCESS CONTROL
//////////////////////////////////////////////////////////////*/

function setAnalyst(address analyst, bool enabled) external onlyOwner {
    if (analyst == address(0)) revert ZeroAddress();
    isAnalyst[analyst] = enabled;
    emit AnalystUpdated(analyst, enabled);
}

/// @notice Step 1 of a two-step handover; protects against typos in the new owner address.
function transferOwnership(address newOwner) external onlyOwner {
    if (newOwner == address(0)) revert ZeroAddress();
    pendingOwner = newOwner;
    emit OwnershipTransferStarted(owner, newOwner);
}

/// @notice Step 2: the new owner confirms.
function acceptOwnership() external {
    if (msg.sender != pendingOwner) revert NotPendingOwner();
    emit OwnershipTransferred(owner, msg.sender);
    owner = msg.sender;
```

```
        pendingOwner = address(0);
    }

    /*//////////////////////////////////////////////////////////////
                                VIEWS
    //////////////////////////////////////////////////////////////*/

    function alertCount() external view returns (uint256) {
        return _alerts.length;
    }

    function getAlert(uint256 alertId) external view returns (Alert memory) {
        if (alertId >= _alerts.length) revert UnknownAlert(alertId);
        return _alerts[alertId];
    }

    /// @return found whether the transaction has been reported, and its alert id if so.
    function alertIdOf(bytes32 txHash) external view returns (bool found, uint256 alertId) {
        uint256 v = _alertIdPlusOne[txHash];
        return (v != 0, v == 0 ? 0 : v - 1);
    }
}
```

B. Team Contribution

Table 11: Team contribution

Team member	USN	Contribution
Rashmi DT	USN0362	Developed the fraud detection and transaction monitoring workflow, implemented hybrid detection using rule-based techniques and ML anomaly detection, integrated the dashboard and handled blockchain anchoring using the Solidity smart contract and Sepolia.
Bhoomika P	USN0090	Contributed to project research, requirement analysis, fraud detection methodology, testing and documentation.
Anita	USN0055	Contributed to system design, dashboard development, testing, validation and project documentation.

C. Geo-Tagged Photograph of the Batch Presentation

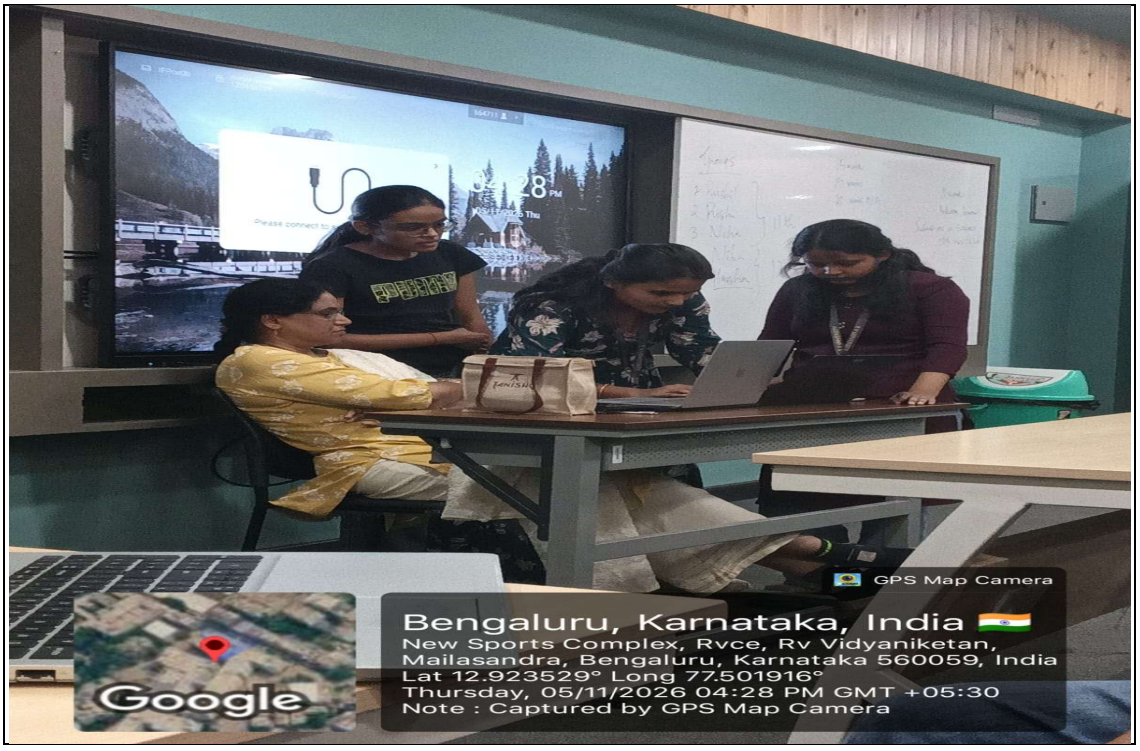


Figure 8: Geo-tagged photograph of the batch during the mini project presentation

Table 12: Details of the geo-tagged photograph

Item	Details
Date and time	05/11/2026 4:28PM
Venue	401 / B Block, RV University, Bengaluru
GPS location (latitude, longitude)	Lat 12.923529 Long 77.501916
Team members present	Rashmi DT (USN0362), Bhoomika P (USN0090), Anita (USN0055)

Item	Details
Evaluated by	Cynthia J

D. Hedera Certifications

Student 1: Rashmi DT (USN0362)

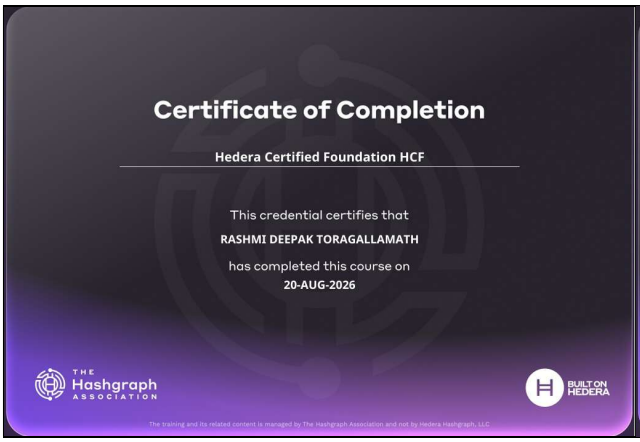


Figure 9: Hedera Part 1 – Student 1



Figure 10: Hedera Part 2 – Student 1

Student 2: Bhoomika P (USN0090)



Figure 11: Hedera Part 1 – Student 2



Figure 12: Hedera Part 2 – Student 2

Student 3: Anita (USN0055)



Figure 13: Hedera Part 1 – Student 3



Figure 14: Hedera Part 2 – Student 3

E. Project Video and GitHub Links

Table 13: Project video and repository links

Item	Details
Video walkthrough link	https://drive.google.com/file/d/1XM5nRSkE7Bwcg5PfBr3fC0w2XQcKWQ9K/view?usp=sharing
GitHub repository link	https://github.com/rashmideepaktoragallamath/onchain-fraud-monitor

F. Conference Paper Draft

Table 14: Details of the conference paper draft

Item	Details
Paper title	Explainable Fraud Detection and Tamper-Evident Transaction Monitoring for On-Chain Finance
Authors	Rashmi DT, Bhoomika P, Anita; Dr. J. Cynthia (Guide)
Target conference / journal	National Conference on Digital Technology & Engineering (NCDTE-2026)
Status	6
Similarity (plagiarism) report	N/A

Explainable Fraud Detection and Tamper-Evident Transaction Monitoring for On-Chain Finance

Rashmi DT
School of Computer Science and Engineering
RV University
Bengaluru, India
rashmideepaktoragallamathbtech24@rvu.edu.in

Bhoomika P
School of Computer Science and Engineering
RV University
Bengaluru, India
bhoomikapbtech24@rvu.edu.in

Anita
School of Computer Science and Engineering
RV University
Bengaluru, India
anitabtech24@rvu.edu.in

Dr.Cynthia J
School of Computer Science and Engineering
RV University
Bengaluru, India
cynthiaj@rvu.edu.in

***Abstract*—Public blockchains let anyone move funds through new pseudonymous wallets, yet no central party screens those transfers for money-laundering behaviour, and the alert records that compliance teams keep can be edited after the fact. This paper presents a prototype monitoring layer that addresses both gaps. A Python engine reads live Ethereum mainnet blocks through a read-only JSON-RPC connection, applies eight transparent rules for known laundering typologies together with an Isolation Forest outlier detector, and merges the signals with a noisy-OR formula into a single 0–100 risk score whose reasons are named. Analysts review one case per wallet in a dashboard whose workflow mirrors a smart contract. Approved alerts are anchored in `RiskRegistry`, a Solidity contract on the Sepolia testnet that stores only a transaction hash, a wallet, a score, a reason bitmask and a review status, protected by role-based access, duplicate detection and final review states. On a simulated data set with 42 injected fraud scenarios, all 42 were detected; alert precision was 0.996 with an exchange allowlist and 0.606 without it. The contract passes 15 test cases, eight of them negative, and real alerts were anchored on Sepolia for 585,087 to 683,007 gas each. The detection results come from simulated fraud and show that the pipeline works as designed; they are not an estimate of accuracy on real laundering.**

***Index Terms*—blockchain, Ethereum, anti-money laundering, fraud detection, Isolation Forest, smart contract, audit trail**

I. Introduction

Ethereum allows anyone to create a wallet without identity checks and to move value within seconds [7]. That openness is valuable for finance, but it also makes it easy to hide the origin of funds by splitting, relaying and returning them through many fresh wallets. Regulators extend anti-money-laundering (AML) duties to businesses that handle virtual assets and ask them to apply a risk-based approach and to monitor transactions [6]. The chain is public, so the raw data needed for such monitoring already exists. What is missing is a tool that turns that data into alerts an analyst can understand, defend, and later prove were not altered.

Three practical problems follow. First, manual review of public chain data is slow, and patterns such as one wallet fanning funds out to dozens of recipients are easy to miss by eye. Second, opaque scores are hard to justify to an auditor, while rule-only engines are easy to evade because their thresholds are guessable. Third, alert logs and review decisions normally live in an ordinary database, so nothing shows that an alert was not deleted or a decision changed afterwards.

This paper makes three contributions:

1) an explainable scoring method that combines transparent AML rules with an unsupervised outlier detector so that every score decomposes exactly into named reasons;
2) `RiskRegistry`, a role-based smart contract that records alerts and review decisions as permanent events, together with an off-chain review workflow that obeys the same state rules; and
3) a working prototype with a live mainnet mode, an analyst dashboard, an automated test suite, a security review and a measured on-chain cost.

Section II gives background and related work, Section III describes the detection pipeline, Section IV the smart contract, Section V the security analysis, Section VI the implementation, Section VII the design rationale, Section VIII the evaluation, and Sections IX and X discuss limitations and conclude.

II. Background and Related Work

A. Laundering patterns on a public ledger

AML practice describes laundering as placement, layering and integration. On a public chain the layering stage leaves recognisable footprints. *Structuring* (or smurfing) splits an amount into several transfers that each stay just under a review limit. *Fan-out* spreads funds from one wallet to many recipients in a short time, and *fan-in* gathers funds from many wallets into one. *Round-tripping* sends funds away and brings them back to the origin. Other signals are very high transfer velocity, unusually large values, freshly created wallets that immediately move large amounts, and contact with wallets on a watchlist. None of these signals proves wrongdoing; exchanges and market makers, for example, legitimately show high velocity and fan-out. A practical system must therefore explain each alert and keep a human in the loop.

TABLE I
COMPARISON WITH RELATED WORK

Work	Approach	Gap addressed here
[1]	GCN on labelled Bitcoin graph	Offline; no alert workflow or audit trail
[2]	Graph features + LightGBM	Supervised; account labels needed; no explained, recorded alerts
[3]	Survey of anomaly detection	No working monitoring layer
[4]	Taxonomy of DeFi fraud methods	Review only; early detection immature
[5]	Isolation Forest	Needs AML features, rules and an explanation layer
This work	Rules + Isolation Forest + on-chain audit trail	Explained live alerts with tamper-evident review

B. Related work

Weber *et al.* [1] applied graph convolutional networks to a labelled Bitcoin transaction graph and showed that graph structure helps identify illicit transactions, but their study is an offline experiment on a single labelled data set. Chen *et al.* [2] detected phishing accounts on Ethereum by extracting graph-based cascade features and training a LightGBM ensemble; the method needs labelled phishing accounts and classifies accounts rather than producing explained alerts with a recorded review. Hassan *et al.* [3] survey anomaly detection in blockchain networks, covering evaluation metrics, requirements and models by blockchain layer, and list open challenges. Luo *et al.* [4] organise DeFi fraud by project life-cycle stage and review statistical, machine-learning and graph methods, noting that early-stage detection is immature because data and reliable ground truth are scarce. Liu *et al.* [5] introduced Isolation Forest, which isolates outliers with random trees and needs no labelled fraud, a property that suits a setting where labels are rare.

The works in Table I either detect well offline or survey the field. None combines real-time monitoring, scores that decompose into readable reasons, and a tamper-evident record of the analyst's decision. Our work targets that combination.

III. Detection Pipeline

A. Architecture

Detection runs off-chain, because scanning every transfer on-chain would be prohibitively expensive, and the chain is used only for a permanent record. Fig. 1 shows the data flow. Blocks are read from an Ethereum node over read-only JSON-RPC and native ETH transfers are stored in SQLite. Features, rules and the outlier detector produce a score with reason codes. Alerts are grouped into one case per wallet. An analyst reviews a case and may queue it for anchoring. A separate script signs and sends the transaction to `RiskRegistry`, so the web server never holds a private key.

TABLE II
RULES, CONDITIONS AND WEIGHTS

Reason	Condition (1-hour window)	w
Structuring	≥ 3 transfers of 9–10 ETH, just under a 10 ETH limit	0.55
Fan-out	one sender, ≥ 15 distinct recipients	0.50
Fan-in	≥ 15 senders into one receiver	0.50
Round-trip	funds return to origin within $\pm 25\%$	0.50
High velocity	≥ 20 transfers by one sender	0.50
Watchlist	counterparty on the watchlist file	0.75
Large value	transfer of ≥ 100 ETH	0.35
New-wallet burst	nonce ≤ 5 and ≥ 20 ETH moved	0.35
ML outlier	Isolation Forest, top 1% most isolated	0.30

B. Ingestion

The ingestion module calls standard Ethereum JSON-RPC methods and stores, for each native ETH transfer, the hash, block, timestamp, sender, receiver, value and sender nonce. A cursor lets a restart continue where it stopped. The client uses https only, redacts the API key from logs, retries with back-off on network errors, accepts compressed responses, and tolerates truncated reads. For repeatable experiments a simulator generates a realistic background of normal transfers and injects labelled fraud scenarios.

C. Features and rules

For each transfer the engine computes behavioural features over a one-hour sliding window. Table II lists the eight rules, the condition each uses in the prototype, and its weight. The conditions map to the typologies of Section II-A. Thresholds are configurable; they were chosen by the team for the prototype and are not calibrated against real compliance data.

D. Isolation Forest

Rules capture known patterns; an unsupervised detector can flag behaviour that no rule describes. We train an Isolation Forest [5] from scikit-learn [8] on the behavioural feature vectors. A transfer whose anomaly score falls in the top 1% is marked as an ML outlier, but only once at least 200 samples are available, so that a nearly empty database does not produce spurious outliers. Because "unusual" is not the same as "wrong", the outlier signal has the lowest weight and cannot raise an alert alone.

E. Explainable risk score

Each signal i carries a weight $w_i \in (0, 1)$, read as how strongly the signal alone suggests risk. Independent signals combine with a noisy-OR rule:

$$S \;=\; 100\Big(1 - \prod_{i \in \mathcal{H}} (1 - w_i)\Big), \tag{1}$$

where $\mathcal{H}$ is the set of signals that fired. The score stays below 100, never decreases when evidence is added, and decomposes exactly into the reasons that produced it. Pattern

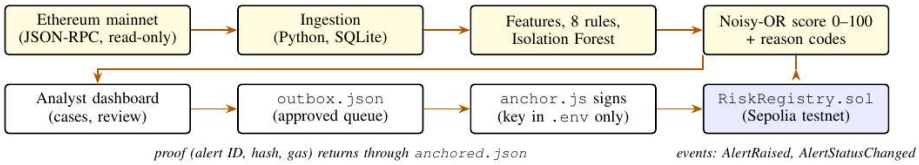


Fig. 1. Architecture. Shaded boxes: off-chain detection (yellow) and the on-chain contract (blue). Only the anchoring script holds a key.

Algorithm 1 Scoring one transfer

$\mathcal{H} \leftarrow \emptyset$; $mask \leftarrow 0$
for all rule r in the eight rules **do**
  if r fires on the window features **then**
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{r\}$; $mask \leftarrow mask \mid (1 \ll bit(r))$
  end if
end for
if Isolation Forest flags the feature vector **then**
  add ML outlier to $\mathcal{H}$ and $mask$
end if
$S \leftarrow \lfloor 100(1 - \prod_{i \in \mathcal{H}}(1 - w_i)) \rfloor$
if $S \geq 50$ **then**
  raise alert $(hash, subject, S, mask)$ with named reasons
end if

rules (weights 0.50–0.55) and the watchlist rule (0.75) can raise an alert alone; context rules (large value, new wallet, outlier; 0.30–0.35) stay below the alert threshold of 50 and need corroboration. Severity is *critical* for $S \geq 80$, *high* for $65 \leq S < 80$ and *medium* for $50 \leq S < 65$. For example, high velocity (0.50), fan-out (0.50) and outlier (0.30) give $100(1 - 0.5 \cdot 0.5 \cdot 0.7) = 82.5$, reported as 82. Algorithm 1 summarises scoring.

F. Cases and review workflow

One wallet can raise many alerts, so the dashboard groups alerts into one *case* per wallet; for fan-in alerts the case belongs to the receiver. An analyst can mark a case Reviewed, Escalated or Dismissed. The allowed moves are exactly those of the contract (Fig. 2): Open may move to any other state, Reviewed may move to Escalated or Dismissed, and Escalated and Dismissed are final. A known exchange can be allowlisted, which dismisses its case and suppresses later behaviour-based alerts for that wallet. Every signal carries a plain-language explanation, why it matters, and a suggested next step, shown next to the evidence so the analyst can justify the decision.

G. Dashboard and API

The dashboard is served by a small standard-library HTTP server bound to localhost. Its read-only endpoints feed the page, and three write endpoints change a case (Table III). The case panel is organised as *Detect* (score, severity, signals), *Explain* (why each signal matters and the next step, with the highest-scoring evidence and recent transfers) and *Prove* (the on-chain alert ID, transaction link and gas once anchored). Wallets whose behaviour resembles an exchange or bot carry a visible label, because high-volume services are the main source of false positives.

TABLE III
LOCAL API USED BY THE DASHBOARD

Endpoint	Purpose
GET `/api/summary`	Tiles, charts, signal counts
GET `/api/cases`	Case list; filters `status`, `min_score`, `q`
GET `/api/cases/{w}`	Case detail, explanations, evidence
POST `.../status`	Reviewed, escalated or dismissed, with note
POST `.../queue`	Queue or unqueue for anchoring
POST `.../allowlist`	Allowlist a wallet and dismiss its case

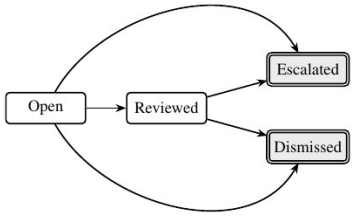


Fig. 2. Review states shared by the dashboard and the contract. Double borders mark final states.

IV. Smart Contract Design

`RiskRegistry` (Solidity 0.8.24) is the on-chain audit trail. It stores no personal data: only a transaction hash, a public wallet address, a score from 1 to 100 and a 16-bit reason mask.

A. Data model and access control

Each alert is a struct holding the monitored transaction hash, the flagged wallet, the analyst who anchored it, two timestamps, the reason mask, the score and the status (an

Listing 1. `raiseAlert` from `RiskRegistry.sol` (event emission shortened).

```
function raiseAlert(bytes32 txHash, address subject,
    uint8 riskScore, uint16 reasonMask)
    external onlyAnalyst returns (uint256 alertId)
{
  if (txHash == bytes32(0)) revert EmptyTxHash();
  if (subject == address(0)) revert ZeroAddress();
  if (riskScore == 0 || riskScore > MAX_RISK_SCORE)
      revert InvalidRiskScore(riskScore);
  if (reasonMask == 0) revert NoReasonGiven();

  uint256 existing = _alertIdPlusOne[txHash];
  if (existing != 0)
      revert AlreadyReported(txHash, existing - 1);

  alertId = _alerts.length;
  _alerts.push(Alert({ txHash: txHash, subject: subject,
      raisedBy: msg.sender,
      raisedAt: uint64(block.timestamp),
      updatedAt: uint64(block.timestamp),
      reasonMask: reasonMask, riskScore: riskScore,
      status: Status.Open }));
  _alertIdPlusOne[txHash] = alertId + 1;
  emit AlertRaised(alertId, txHash, subject,
                   riskScore, reasonMask);
}
```

TABLE IV
REASON BIT POSITIONS (SHARED BY ENGINE AND CONTRACT)

Bit	Reason	Bit	Reason	Bit	Reason
0	Large value	3	Fan-out	6	Watchlist
1	High velocity	4	Fan-in	7	New-wallet burst
2	Structuring	5	Round-trip	8	ML outlier

enum with the four review states). Alerts live in an array; a mapping from transaction hash to alert index lets the contract reject a second report of the same transaction. The deployer is the owner and the first analyst. Only analysts may write alerts or change status. Only the owner manages analysts and a watchlist of flagged addresses, and ownership transfers in two steps (propose, then accept) so that a mistyped address cannot lock the contract.

B. Functions and events

`raiseAlert` validates its inputs and stores the alert; `updateStatus` applies the transition rules of Fig. 2. Listing 1 shows the core of `raiseAlert`. The contract uses custom errors rather than revert strings to save gas and to give precise failure causes, and emits an event for every action (`AlertRaised`, `AlertStatusChanged`, `WatchlistUpdated`, `AnalystUpdated`, ownership events). Events make the audit trail readable by any explorer or indexer without paying for extra storage reads.

C. Reason bitmask

Each reason owns one bit, and the Python engine and the contract use the same positions (Table IV). A mask therefore stores a whole set of reasons in two bytes, and a viewer can decode it without the engine. For instance, mask 266 equals $2^8 + 2^3 + 2^1$ and means ML outlier, fan-out and high velocity.

TABLE V
THREATS AND MITIGATIONS

Threat	Mitigation
Key theft	Secrets only in a git-ignored `.env`; throwaway testnet wallet; only the anchor script signs; mainnet access is read-only
Unauthorised writes	`onlyAnalyst` / `onlyOwner`; two-step ownership transfer
Replay or double reporting	One alert per transaction hash; duplicate reverts
Tampering with history	Alerts are append-only; final states cannot change; every change emits an event
Invalid data on-chain	Score range, non-empty reasons, non-zero hash and wallet enforced
Cross-site requests	Localhost bind; Origin check; custom header on writes
DNS rebinding	Host header check
Script injection	Strict content-security policy; data rendered as text only
Oversized requests	Body size limits; bounded draining of refused bodies
Secret leakage in logs	API key redacted from errors; https only
Bad queue items	Anchor script validates each item, skips invalid ones, never sends a transition the contract would revert, caps 10 transactions per run

V. Security Analysis

Table V lists the main threats considered and the measure that addresses each.

VI. Implementation

The engine is written in Python 3 (pandas, NumPy, scikit-learn) with a standard-library HTTP server and a plain HTML, CSS and JavaScript dashboard. The contract is built and tested with Hardhat and ethers. The dashboard has a simulated mode, a live mode that refreshes every 12 s from mainnet, and a viewer mode for the stored database. It shows summary tiles, alerts over time, active cases by severity, the most common signals, a filterable case list, and a case panel organised as *Detect*, *Explain* and *Prove*.

Anchoring uses a file bridge. When an analyst queues a case, the dashboard writes `data/outbox.json`. The script `npm run anchor` reads it, signs with the deployer key, calls `raiseAlert` and, if the analyst had already decided the case, `updateStatus`, and writes the alert ID, transaction hash and gas used to `data/anchored.json`, which the dashboard merges. The script is idempotent: a second run sends no duplicates. `RiskRegistry` is deployed on Sepolia at `0x1D5a2F95...190924b` (block 12053731).

VII. Design Rationale

Why detection is off-chain. Evaluating windows of behaviour for every transfer would need thousands of storage

reads per block, which is far beyond any reasonable gas budget, and Isolation Forest cannot run in the EVM at all. Only the verdict, a few dozen bytes per alert, is worth paying to store permanently.

Why a file bridge for anchoring. A web server that can sign transactions becomes a high-value target, since any flaw in a page or endpoint could then spend from the wallet. Splitting the roles means the server can only write a queue file, and the only code that touches a key is a short script that validates every item and caps the number of transactions per run. The analyst's decision, not a script, decides what reaches the chain.

Why noisy-OR. A weighted sum can reach 100 from many weak signals and is hard to bound. Noisy-OR keeps the score in range, treats signals as independent pieces of evidence, and lets an analyst reproduce the number by hand from the listed reasons. This is the property regulators and auditors need when they ask why a wallet was flagged.

Why a bitmask and hashes only. A 16-bit mask stores the full set of reasons in one storage word alongside the score, which keeps `raiseAlert` cheap, and storing only hashes and public addresses keeps personal data off a ledger that cannot be erased.

Reproducibility. The code, tests and deployment record are in a public repository. The test commands are `npx hardhat test` (15 contract tests), `npm run test:anchor` (8 tests) and `python -m pytest` (54 tests); the dashboard starts with `python -m monitor.serve --demo` or `--live`, and anchoring runs with `npm run anchor`. Results in Section VIII used a fixed seed (7) for the simulator.

VIII. Evaluation

A. Automated tests

The contract has 15 test cases (Table VI); eight are negative tests that expect a revert. All pass. The anchoring logic has 8 further tests against a fake contract, and the Python engine, dashboard API and security checks have 54 tests, all passing. Testing on Windows exposed a real defect: a rejected request whose body was unread reset the connection, so the client saw a dropped connection instead of the error. The server now discards a bounded amount of the unread body before replying, and a regression test covers every rejection path.

B. Detector on simulated fraud

A simulator generated 3,546 transfers, of which 552 belong to 42 injected scenarios, six for each of seven patterns. The alert threshold was 50 and the random seed 7. Table VII gives the totals and Table VIII the per-pattern outcome. Every one of the 42 scenarios was detected and carried the expected reason code. We measure detection per scenario: 248 alerts fell inside the injected scenarios, so not every injected transfer raised its own alert, and we make no per-transfer recall claim.

The 161 extra alerts without the allowlist come from legitimate high-volume wallets, such as exchanges and routers, whose behaviour resembles layering. The allowlist removes them and lifts precision from 0.606 to 0.996. This result must be read with care: the scenarios were written together with the rules, and the allowlist contains the simulator's own exchange addresses. The figures show that the pipeline works as designed, not how accurate it would be against real laundering.

TABLE VI
Smart contract test cases (N = negative, expects revert)

ID	Scenario	Result
TC-01	Deployer is owner and analyst	Pass
TC-02	Analyst anchors alert; event emitted	Pass
TC-03	Look up alert by transaction hash	Pass
TC-04	N: non-analyst raises alert	Pass
TC-05	N: same transaction reported twice	Pass
TC-06	N: invalid score, empty reasons, zero hash or wallet	Pass
TC-07	Open → Reviewed → Escalated emits events	Pass
TC-08	N: change a final state	Pass
TC-09	N: move back to Open or repeat a state	Pass
TC-10	N: outsider updates; unknown alert ID	Pass
TC-11	Owner adds and removes watchlist entry	Pass
TC-12	N: non-owner edits watchlist; empty reason	Pass
TC-13	Revoked analyst loses write access	Pass
TC-14	Two-step ownership transfer	Pass
TC-15	N: non-owner manages analysts; zero address	Pass

TABLE VII
Detection results on simulated data

Metric	With allowlist	Without
Alerts	249	409
Alerts inside fraud scenarios	248	248
False-positive alerts	1	161
False-positive rate (of transfers)	0.03%	5.38%
Alert precision	0.996	0.606
Scenarios detected	42/42	42/42

C. Live mainnet run

In live mode one run ingested 2,365 real transfers and raised 665 alerts. Mainnet has no ground truth, so we claim no accuracy. Many flagged wallets were high-volume services. The dashboard labels wallets that look like exchanges or bots with "possible exchange/bot" and advises checking ownership before escalating, which confirms that real traffic produces false positives an analyst must resolve.

D. On-chain cost

Table IX lists the gas measured on Sepolia. The figures come from single runs on a testnet and were not tuned.

The `updateStatus` figure is derived from the reported fee of 0.00003534 ETH at a gas price of 1 Gwei.

E. Case studies

Alert #0. A wallet scored 82 (critical) with mask 266: high velocity, fan-out and ML outlier. Its evidence showed

TABLE VIII
PER-PATTERN OUTCOME (SIX SCENARIOS EACH)

Pattern	Detected	Expected reason code
Structuring	6/6	6/6
Fan-out	6/6	6/6
Fan-in	6/6	6/6
Round-trip	6/6	6/6
Watchlisted counterparty	6/6	6/6
New-wallet burst	6/6	6/6
High velocity	6/6	6/6

TABLE IX
GAS MEASURED ON SEPOLIA

Operation	Gas used
Contract deployment (block 12053731)	6,426,712
`raiseAlert`, alert #0 (block 12062466)	683,007
`raiseAlert`, alert #1 (block 12066338)	585,087
`updateStatus` (block 12066339)	$\approx$ 35,300

26 transfers by one wallet within 60 minutes split across 26 recipients. The dashboard also labelled it a possible exchange or bot, so the guidance told the analyst to check ownership first. The block explorer shows the `AlertRaised` event with alert ID 0, score 82 and mask 266.

Alert #1. A wallet scored 68 (high) with mask 260, structuring plus ML outlier, from four alerts of repeated transfers just under the 10 ETH limit. The analyst added the note "repeated sub-threshold transfers" and escalated the case. Anchoring produced `AlertRaised` for alert ID 1 (score 68, mask 260), and a second transaction produced `AlertStatusChanged` from Open to Escalated. Escalated is final, so the decision can no longer be altered.

IX. DISCUSSION AND LIMITATIONS

Keeping detection off-chain and only the verdict on-chain limits cost while making the record permanent and role-controlled. The noisy-OR score is easy to explain but its weights are expert-chosen rather than learned. The limitations are:

- Evaluation uses simulated fraud designed alongside the rules, so high scenario recall is expected; real laundering is more varied.
- Thresholds and weights are hand-chosen and uncalibrated, and the mainnet run has no labels.
- Only native ETH transfers are monitored; ERC-20 tokens, internal transfers and cross-chain flows are not.
- Exchanges raise false positives, and the allowlist is a manual file.
- Anchoring depends on one signing key; a multi-signature scheme would be stronger.
- The contract runs on a testnet and has not had a formal audit.

X. CONCLUSION AND FUTURE WORK

We built and tested a monitoring layer that ingests live Ethereum data, scores behaviour with explainable reasons, supports analyst review, and anchors alerts and decisions in a role-based smart contract. All 42 injected scenarios were detected, all 15 contract tests passed, and real alerts were anchored on Sepolia. Live data showed that exchange traffic causes false alarms, which supports the human-in-the-loop design. Future work includes monitoring ERC-20 and internal transfers, adding graph features and labelled data to calibrate thresholds, batching alerts to reduce gas, requiring multi-signature approval for anchoring, deploying on a low-cost layer-2 network, and sending alerts to compliance tools by webhook.

REFERENCES

[1] M. Weber *et al.*, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019, arXiv:1908.02591.

[2] W. Chen, X. Guo, Z. Chen, Z. Zheng, and Y. Lu, "Phishing scam detection on Ethereum: Towards financial security for blockchain ecosystem," in *Proc. 29th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2020, pp. 4506–4512.

[3] M. U. Hassan, M. H. Rehmani, and J. Chen, "Anomaly detection in blockchain networks: A comprehensive survey," arXiv:2112.06089, 2022.

[4] B. Luo, Z. Zhang, Q. Wang, A. Ke, S. Lu, and B. He, "AI-powered fraud detection in decentralized finance: A project life cycle perspective," arXiv:2308.15992, 2023.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[6] Financial Action Task Force, "Updated guidance for a risk-based approach to virtual assets and virtual asset service providers," Oct. 2021. [Online]. Available: https://www.fatf-gafi.org

[7] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[10] Ethereum Foundation, "Solidity documentation." [Online]. Available: https://docs.soliditylang.org

G. How to Run the Prototype

Code repository: https://github.com/rashmideepaktoragallamath/onchain-fraud-monitor

Table 15: Commands to run the prototype

Task	Command (project folder)
Install	pip install -r requirements.txt and npm install
Contract tests	npx hardhat test (15 tests); npm run test:gas (gas report)
Python and dashboard tests	python -m pytest (54 tests)
Anchoring tests	npm run test:anchor (8 tests)
Evaluation	python -m monitor.evaluate
Dashboard, simulated data	python -m monitor.serve --demo, then open http://127.0.0.1:8000
Dashboard, live mainnet	python -m monitor.serve --live (needs MAINNET_RPC_URL in .env)
Anchor queued cases	npm run anchor (needs SEPOLIA_RPC_URL and DEPLOYER_PRIVATE_KEY in .env)